Assignment-5

* Problem Statement:

Department of CSE has student club named Pinnacle club. Students of 2nd, 3rd & final year can be granted membership on request. Similarly, one may cancel the membership of club. First node is reserved for president of club & last node is reserved for secretary.

Write C program to maintain club member's info using Singly linked list, Store PRN & name.

Write func^n to: a) Add & Delete Members as well as president.
b) Comput; total no. of members of club.
c) Display Member
d) String of two linked list
e) Merging of Linked list
f) Reversing using pointers

* Objective: i) To understand concept of SLL & dynamic memory allocation using pointer in C.

ii) To perform insertion, deletion, count nodes, sort & reverse operat^n on SLL.

iii) To develop menu-driven app for student club.

* Test Condition:

TC	I/P Operation	Expected O/P
TC_1	Add President	President's PRN & name stored in successfully as first node
TC_2	Add Secretary	Secretary PRN & Name a stored in last node.
TC_3	Add regular club member	Student record added t linked list
TC_4	Add multiple members	All records are stored in LL in order
TC_5	Display Empty club list	Msg : no student record
TC_6	Delete existing regular member	The selected student recor is deleted.
TC_7	Display all members	President, Secretary
TC_8	Delete non-existing PRN	Student not found
TC_9	Count club member	Total count is displayed.

* Conclusion:

~~This program successfully implements polynomial creation, deletion & addition using~~

The program demonstrate dynamic memory allocation & efficient manipulation of linked data manipulation of linked data structures operation such as insertion, deletion, traversal, sorting & reverssing were successfully implemented.

* FAQs:

Q1. Why is linked list preffered over an array in this application?

A- i) Dynamic Size: members can be added or removed easily.

ii) Easy insertion & deletion

iii) Memory Efficienat: Memory is allocated as needed.

iv) Flexible Management: President & Secretary can be add or delete.

Q2 What info is stored in each node of circular linked list?

A- i) Data: Student PRN & name.
ii) Pointer: Address of next node, which points back to ~~first~~ node. That is next.

Q3. What is SLL?

A- It is linear data structure in which node contains data & pointer to next node.

- The last node points to the null & indicates the end of list.

Q4. What information is stored in each node?

A- Each node contains the student PRN & name & pointer to the next node.

Q5. What is dynamic memory allocation?

A- Dynamic memory allocation is a process of allocating memory to variable at the time of program execution.

- Malloc() & Calloc() is ~~use~~ used for it.

Q6. Which function is used to release allocated memory?

A- The free() func^{n} is used to release an allocated memory in C.

Q7. What is time complexity of Inserting node at beginning?

A- For insertion at beginning, the time complexity is O(1).

Q8 What is time complexity of searching a student?

A- O(n)

Q9 How is linked list reversed?

A- Linked list can be reversed by changing the direction of pointers.

It is done by using prev, curr & next variable pointers.

Q10. Which Sorting technique can be used in linked list?

A- Bubble sort can be used in linked list.

PRACTICAL 5

DATA STRUCTURE

Pinnacle Club Management System

Singly Linked List in C

Program Code and GDB Input/Output Screenshot

Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct Node {
    long long prn;
    char name[50];
    struct Node *next;
};
struct Node *head = NULL;
/* Create a new node */
struct Node* createNode(long long prn, char name[]) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    newNode->prn = prn;
    strcpy(newNode->name, name);
    newNode->next = NULL;
    return newNode;
}
/* Count members */
int countMembers(struct Node *temp) {
    int count = 0;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }
    return count;
}
/* Display members */
void display(struct Node *temp) {
    if (temp == NULL) {
        printf("Club is empty!\n");
        return;
    }
    printf("\n--- Pinnacle Club Members ---\n");
    printf("PRN\t\tName\t\tPosition\n");
    int count = countMembers(temp);
    int i = 1;
    while (temp != NULL) {
        printf("%lld\t\t%s\t\t", temp->prn, temp->name);
        if (i == 1)
            printf("President");
        else if (i == count)
            printf("Secretary");
        else
            printf("Member");
        printf("\n");
        temp = temp->next;
        i++;
    }
}
/* Add president at beginning */
void addPresident() {
    long long prn;
    char name[50];
    printf("Enter President PRN: ");
    scanf("%lld", &prn);
    printf("Enter President Name: ");
    scanf(" %49[^\n]", name);
    struct Node *newNode = createNode(prn, name);
    newNode->next = head;
    head = newNode;
    printf("President added successfully!\n");
}
/* Add secretary at end */
void addSecretary() {
    long long prn;
    char name[50];
    printf("Enter Secretary PRN: ");
    scanf("%lld", &prn);
    printf("Enter Secretary Name: ");
    scanf(" %49[^\n]", name);
    struct Node *newNode = createNode(prn, name);
    if (head == NULL) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }
    printf("Secretary added successfully!\n");
```

```
}
/* Add ordinary member before secretary */
void addMember() {
    long long prn;
    char name[50];
    printf("Enter Member PRN: ");
    scanf("%lld", &prn);
    printf("Enter Member Name: ");
    scanf(" %49[^\n]", name);
    struct Node *newNode = createNode(prn, name);
    if (head == NULL) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL && temp->next->next != NULL)
            temp = temp->next;
        if (temp->next == NULL) {
            temp->next = newNode;
        } else {
            newNode->next = temp->next;
            temp->next = newNode;
        }
    }
    printf("Member added successfully!\n");
}
/* Delete president */
void deletePresident() {
    if (head == NULL) {
        printf("Club is empty!\n");
        return;
    }
    struct Node *temp = head;
    head = head->next;
    printf("Deleted President: %s\n", temp->name);
    free(temp);
}
/* Delete secretary */
void deleteSecretary() {
    if (head == NULL) {
        printf("Club is empty!\n");
        return;
    }
    if (head->next == NULL) {
        printf("Deleted: %s\n", head->name);
        free(head);
        head = NULL;
        return;
    }
    struct Node *temp = head;
    while (temp->next->next != NULL)
        temp = temp->next;
    struct Node *last = temp->next;
    printf("Deleted Secretary: %s\n", last->name);
    temp->next = NULL;
    free(last);
}
/* Delete member by PRN */
void deleteMember() {
    long long prn;
    if (head == NULL) {
        printf("Club is empty!\n");
        return;
    }
    printf("Enter PRN to delete: ");
    scanf("%lld", &prn);
    struct Node *temp = head;
    struct Node *prev = NULL;
    while (temp != NULL && temp->prn != prn) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("PRN not found!\n");
        return;
    }
    if (prev == NULL)
        head = temp->next;
    else
        prev->next = temp->next;
    printf("Deleted member: %s\n", temp->name);
    free(temp);
}
/* Reverse linked list using pointers */
void reverseList() {
    struct Node *prev = NULL;
    struct Node *current = head;
    struct Node *nextNode = NULL;
```

```
    while (current != NULL) {
        nextNode = current->next;
        current->next = prev;
        prev = current;
        current = nextNode;
    }
    head = prev;
    printf("Linked list reversed!\n");
}
/* Sort list by PRN using bubble sort of data */
void sortList(struct Node *start) {
    if (start == NULL)
        return;
    struct Node *i, *j;
    long long tempPRN;
    char tempName[50];
    for (i = start; i->next != NULL; i = i->next) {
        for (j = start; j->next != NULL; j = j->next) {
            if (j->prn > j->next->prn) {
                tempPRN = j->prn;
                j->prn = j->next->prn;
                j->next->prn = tempPRN;
                strcpy(tempName, j->name);
                strcpy(j->name, j->next->name);
                strcpy(j->next->name, tempName);
            }
        }
    }
}
/* Merge two lists by connecting pointers */
struct Node* mergeLists(struct Node *first, struct Node *second) {
    if (first == NULL)
        return second;
    struct Node *temp = first;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = second;
    return first;
}
/* Read a separate linked list */
struct Node* createList() {
    struct Node *newHead = NULL;
    struct Node *tail = NULL;
    int n, i;
    long long prn;
    char name[50];
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter PRN: ");
        scanf("%lld", &prn);
        printf("Enter Name: ");
        scanf(" %49[^\n]", name);
        struct Node *newNode = createNode(prn, name);
        if (newHead == NULL)
            newHead = newNode;
        else
            tail->next = newNode;
        tail = newNode;
    }
    return newHead;
}
/* Free all nodes */
void freeList(struct Node *temp) {
    while (temp != NULL) {
        struct Node *nextNode = temp->next;
        free(temp);
        temp = nextNode;
    }
}
int main() {
    int choice;
    struct Node *list2 = NULL;
    do {
        printf("\n===== PINNACLE CLUB MENU =====\n");
        printf("1. Add President\n");
        printf("2. Add Member\n");
        printf("3. Add Secretary\n");
        printf("4. Delete President\n");
        printf("5. Delete Member by PRN\n");
        printf("6. Delete Secretary\n");
        printf("7. Count Members\n");
        printf("8. Display Members\n");
        printf("9. Sort Members by PRN\n");
        printf("10. Create Second List\n");
        printf("11. Merge Two Lists\n");
        printf("12. Reverse List\n");
```

```
        printf("13. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                addPresident();
                break;
            case 2:
                addMember();
                break;
            case 3:
                addSecretary();
                break;
            case 4:
                deletePresident();
                break;
            case 5:
                deleteMember();
                break;
            case 6:
                deleteSecretary();
                break;
            case 7:
                printf("Total members: %d\n",
                       countMembers(head));
                break;
            case 8:
                display(head);
                break;
            case 9:
                sortList(head);
                printf("List sorted by PRN!\n");
                break;
            case 10:
                if (list2 != NULL) {
                    printf("Second list already exists. "
                           "Merge it before creating another.\n");
                } else {
                    printf("Create Second Linked List:\n");
                    list2 = createList();
                }
                break;
            case 11:
                if (list2 == NULL) {
                    printf("Create the second list first!\n");
                } else {
                    head = mergeLists(head, list2);
                    list2 = NULL;
                    printf("Lists merged successfully!\n");
                }
                break;
            case 12:
                reverseList();
                break;
            case 13:
                freeList(head);
                freeList(list2);
                head = NULL;
                list2 = NULL;
                printf("Exiting program. Goodbye!\n");
                break;
            default:
                printf("Invalid choice! Try again.\n");
        }
    } while (choice != 13);
    return 0;
}
```

GDB / Program Input and Output Screenshot

